

Plan-Driven Process Models II — Increments, the Spiral and Choosing

Three ways of moving the bad news earlier, then a heuristic for choosing — applied part by part to SANTE-BASSAM

Session 4 of 28 · Friday 18 September 2026 · 10:00 – 11:20 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 4 OF 28

Plan-Driven Process Models II — Increments, the Spiral and Choosing

Friday, 18 September 2026 · 10:00 – 11:20 AM · Unit 1 · Foundations and Process

LEARNING OBJECTIVES

- ✓ Distinguish incremental development from incremental delivery, and say what each lets the team learn before the end.
- ✓ Order increments by risk rather than by ease of building, and justify the order for SANTE-BASSAM.
- ✓ Describe the four spiral quadrants, compute risk exposure, and use the largest unretired risk to decide what to build next.
- ✓ Select a process model for a given project and defend it against a named alternative on three project characteristics.
- ✓ Draw the line in SANTE-BASSAM between what is specified once and what is delivered iteratively.

FROM SESSION 3

Waterfall found the wrong search requirement in week 12 at a cost of nine team-days. The V-model found a hole in the test plan in week 2 — but still met the clerk in week 12. Today's three models are three different ways of **meeting the clerk earlier**, and the session ends with how to choose.

Thought experiments, not history.

Every plan in this session — the increments INC 1–4, the deliveries D1–D3, the spiral's loops — is what that process *would* do with SANTE-BASSAM. Nothing has been built. The labels are temporary; the team's own process, and its story numbers, are decided in Session 5.

Applying Session 3. Six people wrote tests for homework, in six documents. A matrix that exists in six copies is not a matrix.

WHAT HAPPENS

1. **Merge.** One matrix on the board: UR-1 to UR-4 across the four test levels. Where two people wrote the same test differently, keep the one whose failure would be more informative and say why. **Six minutes.**
2. **Own it.** Every test in the matrix gets **one name** against it — the person who will write it when the time comes. Spread the levels: nobody owns only unit tests. **Five minutes.**
3. **Count the empties.** Any cell still blank is a defect route. List them; they are the first thing Session 11 will ask about. **Four minutes.**

Output: `tests/traceability.md` committed, with owners and the empty-cell list.

Next session's slot: three capabilities the post needs first, described in ordinary language — before Session 5 turns them into stories.

1 · Incremental development

ONE RULE

Build the system as a sequence of working, integrated slices — each with its own small specify, design, build, test cycle — but **release nothing until the whole is complete**. The team learns from integration. The customer sees nothing until the end.

What it fixes

Integration is no longer one event in week 11. Each slice integrates with the last, so interface mismatches surface early and small.

What it does not fix

Requirements risk. If the clerk needed search by the mother's name, the team still learns it at the end — she has not touched the system.

The ordering rule

Risk first, then value. The slice that answers your scariest technical question comes first — not the easiest or the prettiest.

APPLIED TO SANTE-BASSAM

Four increments, risk-first — and nothing shipped until week 14

Increment	Weeks	What is built	Why here
INC 1	3–5	Offline capture and sync — PatientStore on both devices, SyncQueue, ServerAPI acknowledgement	The scariest technical question. If it cannot work, the project changes shape
INC 2	6–8	Registry and search — full Patient, SearchService ranking, DeskUI	Depends on INC 1's store; the largest remaining unknown
INC 3	9–11	Appointments and visits — book, reschedule, outcome, the tablet flow	Well understood; safe to leave until the risky parts are retired
INC 4	12–13	The report — ReportGenerator, and IT-03 closes the gap from Session 3	Depends on everything above it

Learned by week 8 — waterfall would learn it in week 11

The sync contract was wrong: the desk needs the full record back on acknowledgement, not just the id. Two hours, not two days.

Card numbers are not unique — the post restarted numbering in 2023. Card number becomes an attribute, never the key.

The tablet ranks 10,000 records in 4 s against the laptop's 0.6 s.

Fixed with a prefix index, inside the increment.

Still unknown in week 13

Whether the clerk will use it.

Whether the district officer trusts the report.

Whether “mother's name and village” is even the right search.

...because **no clerk has touched the system**. The chair is empty from week 1 to week 14.

CARRY FORWARD

Incremental development is a way of **building**. It changes what the team learns from itself. It does not change what the team learns from the customer, because the customer is not in the loop.

2 · Incremental delivery

ONE RULE

Each increment is not merely integrated but **deployed to real users and put into service**. The first delivery is the highest-priority core; every later one adds to a system already in daily use.

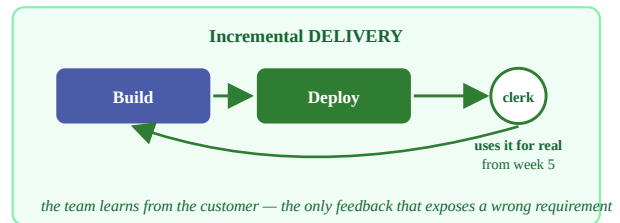
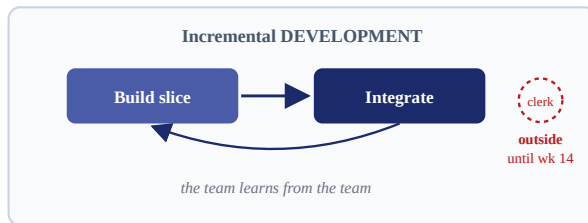


Figure 1. The two diagrams differ by one box. That box is the difference between learning that your modules fit together and learning that you built the wrong thing.

The cost

Every delivery is a release: install, migrate data, write a training note, support it, and provide a way back if it fails. Four deliveries cost more than one.

What it buys

The ability to be wrong cheaply. A wrong requirement in delivery 1 is corrected in delivery 2, with ten weeks left and the code still small.

The discipline

Each delivery must be a **usable whole**. “Registration without search” is not deliverable — the clerk would register patients she can never find again.

APPLIED TO SANTE-BASSAM

The same four increments, re-cut as three deliveries

	Week	What is deployed	After this, the clerk or nurse can...	What real use will answer
D1	6	Register and find, on the desk laptop only. The register migrated. No tablet, no report	...stop scanning the book to find a returning patient	Is “mother’s name and village” the right search? This is the question the whole project turns on
D2	10	Add the tablet: offline outreach registration and sync	...register a family in a village and see them at the desk that evening	Does the CHW trust sync enough to stop carrying paper?
D3	13	Add appointments, visit outcomes, the follow-up list and the report	...retire the notebook, and submit the month without re-counting	Does the officer sign it?

Note what changed from the increment plan. INC 1 was the technical spike, which is not deliverable — nobody can use a sync path. So the spike stays a spike, and the first thing the clerk touches is a whole, small, useful desk application in week 6 rather than a complete system in week 14.

3 • The spiral: process driven by risk

ONE RULE

The largest unretired risk decides what you build next. The spiral (Boehm, 1988) is a meta-process: each loop passes through four quadrants, and distance from the centre is cumulative cost.

What is this loop for? What is fixed, and what are the two or three ways of doing it?

Prototype, benchmark, spike. **Quantitative:** for each risk estimate probability **P** and loss **L** in team-days.

Q3 · Develop and verify

Any local process — a small V, a couple of increments. The spiral does not care how you build this part.

Q4 · Plan the next loop

Review with stakeholders. Commit, or stop. Then re-sort the risk register.

risk exposure $RE = P \times L$

the risk with the largest RE is what the next loop works on — even when it is the least visible thing the team could build

APPLIED TO SANTE-BASSAM

The risk register that drives the loops, and the four loops themselves

Risk	What could go wrong	P × L	RE
R1	Offline sync cannot be made reliable on the tablet	0.4 × 60	24
R2	The clerk cannot find returning patients — no card, duplicates	0.5 × 30	15
R3	The district officer distrusts the report and keeps hand-counting	0.5 × 20	10
R4	Power failures corrupt the local database	0.3 × 25	7.5
R5	The team cannot reach the post often enough for feedback	0.4 × 15	6
R6	Legal constraints discovered late	0.2 × 20	4

R1 wins loop 1 — and it is the least visible thing the team could build. R4 rides along: the same spike retires both.

Loop	The scariest question	Q2 — how the risk would be resolved	Q4 — a plausible outcome
1 · wk 2–4	Can the tablet work offline?	Thinnest register-offline-then-sync path for two candidate stores; pull the plug twenty times each. One loses nothing once journaling is on; one loses 1.8 s of saves	R1 drops to P 0.05. R4 retired as well. Next: R2
2 · wk 5–8	Can the clerk find a patient with no card?	Paper prototype, 50 real cards, 10 lookups. Exact-name search fails 4 of 10; fuzzy name with village and birth year succeeds 10 of 10 — and the clerk volunteers “I always ask the mother’s name”	R2 retired. New risk R7: duplicate entries, RE 5. Next: R3
3 · wk 9–12	Will the officer trust the report?	Mock report from one real month. “What would you check before signing?” She hand-checks two villages — so the export is her audit trail	R3 retired. Next: R5, hand-over
4 · wk 13–14	Can the post run it without us?	Clerk registers ten real patients, nurse books five appointments, team absent. Risk found: nobody at the post can restart sync after a failure	One-page operations sheet and a “sync now” button that reports in plain language. Release 1.0

LOOK AT LOOP 2 AGAIN

“I always ask the mother’s name” is the same sentence waterfall heard in **week 12** for nine team-days. The spiral heard it in **week 5**, for one afternoon and fifty paper cards, before a line of search code existed. The spiral did not make the team smarter. It made them ask the scariest question first, every four weeks, and spend money on the answer.

Carry forward — and this one outlives plan-driven process entirely

Every modern process, agile included, inherits the spiral’s rule: **the largest unretired risk decides what you build next**. Teams that iterate without a risk register are not doing the spiral. They are doing waterfall in small pieces.

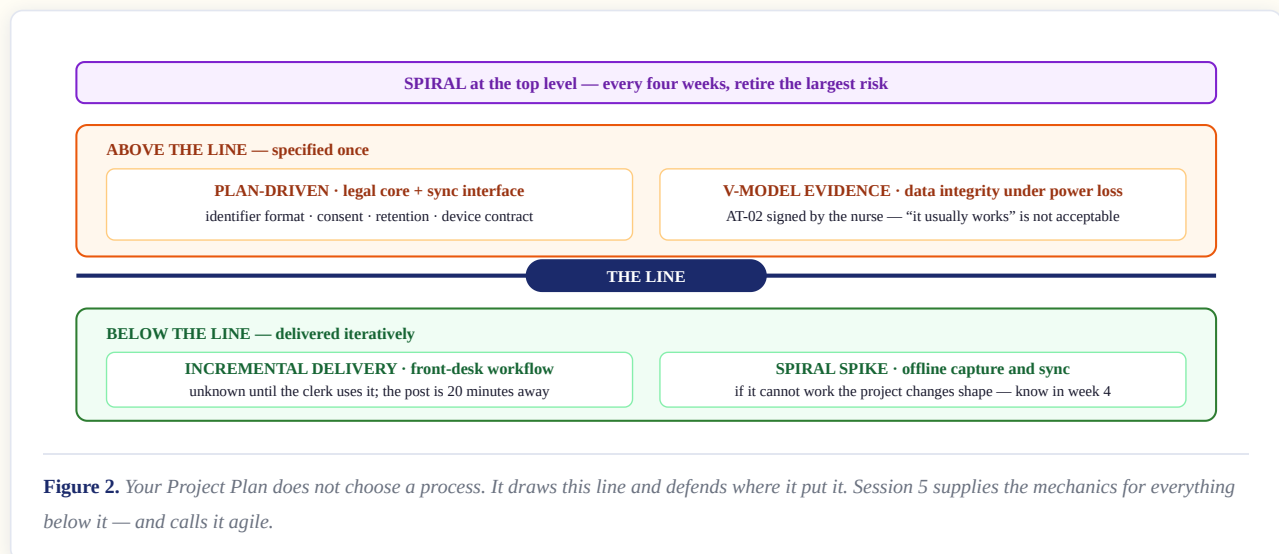
4 · Choosing

Read the project's dominant characteristic; it points at a process. Real projects mix — a hospital may certify its clinical-rules module under a V while running its appointment front end in two-week sprints. **Choosing one process for the whole project because it is the one you know is a failure of judgement, not a simplification.**

Dominant characteristic	Points at
Requirements stable and externally fixed — regulation, contract, standard	Waterfall or V
Safety-critical; certification evidence required	V-model
Requirements genuinely unknown until users touch the system	Incremental delivery, agile
Novel technology; major technical uncertainty	Spiral — spike first
Large distributed team, several organisations, fixed interfaces	Plan-driven between, agile within
Small co-located team with an accessible customer	Agile — Session 5

APPLIED TO SANTE-BASSAM

One project, four processes — the position your Project Plan will defend



SESSION 4 TAKEAWAY

Incremental development moves integration risk earlier; incremental delivery moves requirements risk earlier, and it is the only one of the five that does, because it is the only one that puts the customer inside the loop. The spiral moves whichever risk is biggest right now, by insisting you price it before you build anything. If you remember one sentence from these two sessions: **a process model is a decision about when the team is allowed to find out it was wrong.**

5 · In-class exercise · process selection

Twelve minutes in three pairs, six minutes defending. Four projects; your pair takes two. Ask the four questions: how volatile are the requirements? how expensive is a late defect? what evidence must the project produce? what feedback exists before release?

The four projects

Project	The situation	The trap
A · Crop-spraying drone	Flight-control software for an agricultural drone over cocoa plantations. A fault could injure workers. Civil-aviation certification required before commercial use	—
B · Mobile-money savings app	Fintech start-up in Abidjan. Three competing hypotheses about what customers want; eight months of funding	—
C · COBOL payroll replacement	Twenty-year-old system, 4,000 employees. The new system must match the old exactly, quirks included. Parallel run for one full payroll cycle	The requirements “exist” — only as undocumented behaviour in old code
D · Handwriting reader	Reads agricultural extension officers’ field notes. Nobody knows whether the needed accuracy is achievable	Feasibility risk, not requirements risk. Sprints without a kill criterion are not risk management

PROJECT

Chosen model :

Three characteristics that drove it:

Named alternative rejected, and the specific reason:

The largest risk my chosen process leaves unaddressed:

Then, together, one question: which of the four is most like SANTE-BASSAM — and where does the analogy break?

6 · Homework · individual · due before Session 5 (Tuesday 22 September)

Download **CSC4305_HW04_Template.md** from **CSC_4305_SWE-Assignments**, fill it in — type between the **<!-- ANSWER -->** markers, do not delete the questions or the tables. **Rename** it **CSC4305_HW04_FirstName_LastName.md** and **commit** it to **/homework/S04/** in **your own private repository**.

TEMPLATE

Markdown only. A **.md** file — not **.docx**, not a PDF, not a Google Docs link, not **.pages** or **.odt**. Writing it in Word is fine — convert the **.docx** for free at word2md.com and commit the Markdown it hands back. Full instructions are in the *Read this first* box at the top of the template.

Q1 Your own delivery plan. Three to five deliveries for SANTE-BASSAM with a target week. For each: what is deployed; one sentence saying “after this delivery the clerk / nurse / CHW / officer can...”; the acceptance test that must pass; and the one question real use will answer. Use \$2 as the standard, **not** as the answer — yours should differ.

Test every delivery against one question: *could the post actually use this on its own for three weeks? If not, it is an increment, not a delivery.*

Q2 The first delivery, in detail. For your D1 only, write three short pieces: a release note (what is in, what is known broken, who to call), a one-page training sheet the clerk could follow alone, and a rollback plan (the exact steps back to paper).

Hand-drawn screenshots on the training sheet are fine and often better.

Q3 Price two risks. Add two risks to the register in §3 that are not there — real ones, for this project. Estimate P and L for each, compute RE, and say where they would fall in the ordering. Then: which loop would tackle yours, and what would that loop actually build?

Q4 Compare, in 250 to 400 words. Against the four-increment plan in §1: what does your delivery plan let the team learn, and when, that the increment plan does not? What does each delivery cost that an increment does not? Be specific to SANTE-BASSAM — name the week and the

person.

Q5 Draw your own line. Redraw Figure 2 for SANTE-BASSAM, moving at least one thing from where this document put it. Defend the move in two sentences. If you would not move anything, say which item you considered moving and why you left it.

Q6 Reading. Sommerville, *Software Engineering* 10th ed., §2.1–2.3. Optional: Boehm (1988), §1–3.

Where this goes next. Everything below the line needs machinery: a unit of work small enough to finish and concrete enough for the clerk to judge, a way of deciding how much fits in a fortnight, and a fixed date on which she touches the software. That is Tuesday, and it is where the project starts moving.

Key terms

incremental development

incremental delivery

usable whole

spiral model

risk register

risk exposure $P \times L$

unretired risk

spike

requirements volatility

process tailoring

the line

Readings and references

- Sommerville, I. *Software Engineering*, 10th ed. — Chapter 2, §2.1–2.3.
- Boehm, B. “A Spiral Model of Software Development and Enhancement”, *IEEE Computer* 21(5), 1988.
- Royce, W. “Managing the Development of Large Software Systems”, Proc. IEEE WESCON, 1970.
- Forsberg, K. & Mooz, H., NCOSE, 1991 — the V-model.
- Pressman, R. & Maxim, B. — Chapter 2.